CNN, LSTM and Transformer on Short-Text Intent Classification

AIGC 5500 Advanced Deep Learning, Midterm Project, Group 01: Ranjay Salmon, Ankit Yadav, Ripan Deep Kour, Teja Ekkurthi

1 Problem and data

We asked how convolutional, recurrent and attention-based models compare at classifying the intent of a short utterance when they share the same split, optimizer, training budget and tuning effort, and whether a bag-of-words baseline already does the job. SNIPS is our primary dataset: voice-assistant commands in 7 balanced intents. ATIS is our second dataset: flight-booking queries in 16 intents, where the “flight” class alone is 606 of the 819 test rows and the rarest class has one test row.

Dataset	Train	Validation	Test	Classes	Vocabulary
SNIPS	9983	2139	2140	7	3671
ATIS	3819	819	819	16	590

Our first pipeline split the raw data. When we checked, about 2% of SNIPS test rows and 8–9% of ATIS test rows also appeared word for word in training, which is leakage. We now remove repeated utterances before a stratified 70/15/15 split, confirmed zero overlap between splits, and drop classes with fewer than 10 unique examples so that every split contains each class. The split seed is fixed, so seeds vary only model initialisation and shuffling. We lowercase, tokenize by regex, build the vocabulary from training text only, and pad or cut to 35 tokens.

2 The three models and why

Each model starts with a 128-dimension embedding trained from scratch. **TextCNN** runs convolutions over windows of 2, 3 and 4 words, so it responds to local phrases wherever they occur. **TextRNN** is a one-layer bidirectional LSTM with masked mean pooling, so it builds state along the sequence. **TextTransformer** is a two-layer, four-head encoder with sinusoidal positions and padding masks, so any word can look at any other directly. Parameter counts are 588,095, 735,879 and 735,751 on SNIPS, within a factor of 1.3.

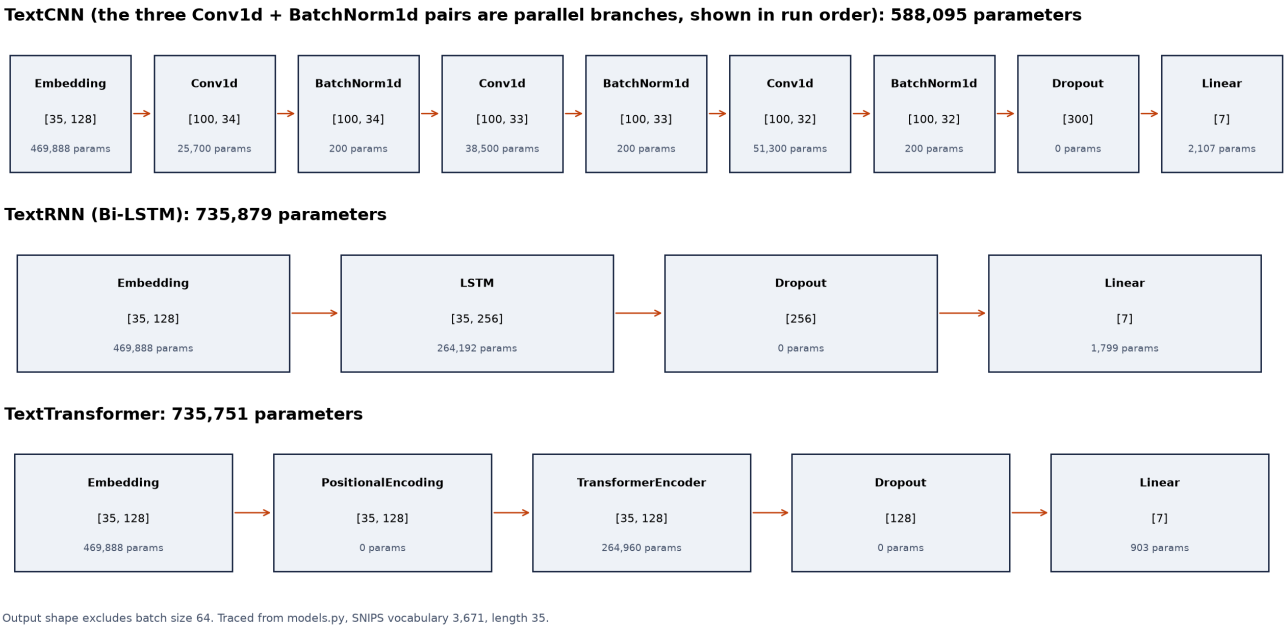


Figure 1. Layer shapes recorded by running a batch through the classes in models.py (diagrams.py).

3 Baseline

Our baseline is logistic regression on log-scaled word counts. It reads the same token ids as the deep models and shares their optimizer, 20-epoch budget and tuning. It has only 25,704 parameters on SNIPS, far below the deep models, because being a much simpler model is its purpose. Every family tuned the same grid of 10 settings: learning rates 0.0003, 0.001, 0.003, 0.01 and 0.03, each with weight decay 0 and 0.01, at seed 42, chosen by validation macro F1. Ablations keep their family's settings.

Family	SNIPS lr, decay	SNIPS val F1	ATIS lr, decay	ATIS val F1
Baseline	0.03, 0.0	0.9882	0.03, 0.0	0.7671
CNN	0.01, 0.01	0.9864	0.003, 0.01	0.8712
Bi-LSTM	0.01, 0.01	0.9892	0.003, 0.0	0.8897
Transformer	0.001, 0.01	0.9850	0.001, 0.0	0.8409

Our first grid stopped at a learning rate of 0.001. The baseline then underfit, with ATIS validation F1 of 0.21 and test F1 of 0.18, and that learning rate was the best available for every family, so we widened the grid for all of them. The baseline's best setting is still the top of the grid, so it might gain a little from going higher.

4 Results

Table: SNIPS test macro F1, every seed.

Model	Seed 42	Seed 123	Seed 999	Mean	Spread
Baseline	0.9812	0.9822	0.9827	0.9820	0.0014
CNN	0.9842	0.9822	0.9814	0.9826	0.0028
Bi-LSTM	0.9807	0.9803	0.9818	0.9809	0.0015
Transformer	0.9775	0.9738	0.9765	0.9759	0.0037

On SNIPS the CNN (0.9826) and baseline (0.9820) can't be separated: their seeds overlap. The Bi-LSTM (0.9809) is close behind. All three Transformer seeds fall below every seed of the other models. Our explanation is that intents here are signalled by keywords such as "play" or "weather", which a word-count model or a local filter finds directly. In the Transformer's seed-42 confusion matrix, the largest group of errors is SearchCreativeWork rows predicted as PlayMusic or SearchScreeningEvent, classes that can share titles and names. This mechanism is our interpretation; we did not test it directly.

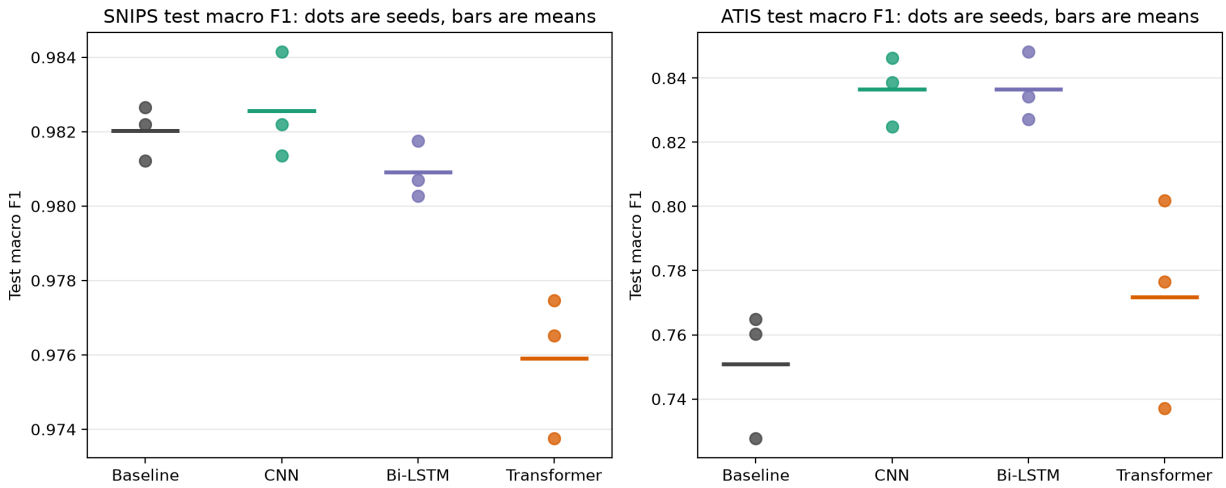


Figure 2. Test macro F1 on both datasets; dots are seeds and bars are means. The y-axes differ.

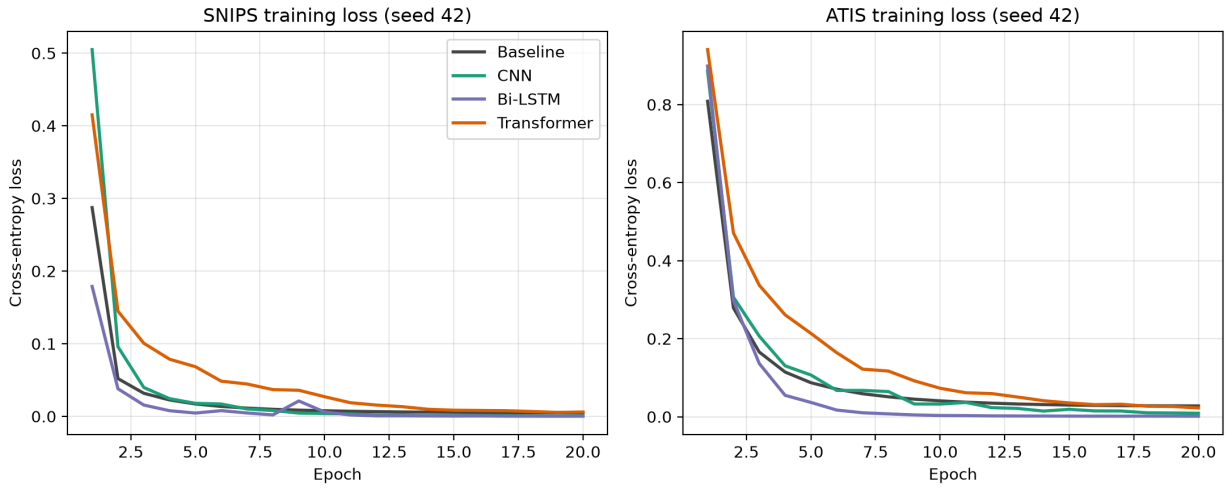


Figure 3. Training loss at seed 42. The Transformer falls most slowly on both datasets.

5 Ablations

Table: SNIPS test macro F1 with one component removed, every seed.

Model	Seed 42	Seed 123	Seed 999	Mean	Spread
CNN, no BatchNorm	0.9827	0.9795	0.9785	0.9803	0.0042
LSTM, one direction	0.9803	0.9803	0.9759	0.9789	0.0044
Transformer, no positions	0.9789	0.9765	0.9752	0.9769	0.0037

Component removed	Standard	Ablated	Change	Per seed (42, 123, 999)
CNN: BatchNorm	0.9826	0.9803	-0.0023	-0.0014, -0.0027, -0.0028
LSTM: backward pass	0.9809	0.9789	-0.0021	-0.0004, +0.0000, -0.0058
Transformer: positions	0.9759	0.9769	+0.0010	+0.0014, +0.0028, -0.0013

Removing BatchNorm lowered the CNN on all three seeds, a small but consistent change, although the standard CNN ran on the CPU and the ablated CNN on the GPU, so the device differs as well as the component. Removing the LSTM's backward pass made almost no difference on two seeds and a larger one on the third, so we can't call it an effect; this ablation also removes about 133,000 parameters, so it changes capacity as well as direction. Removing positions did not hurt the Transformer on average, which fits keyword-driven intents where word order adds little. With three seeds we treat all of these as indications only.

6 Second dataset

Table: ATIS test macro F1, every seed.

Model	Seed 42	Seed 123	Seed 999	Mean	Spread
Baseline	0.7649	0.7602	0.7277	0.7509	0.0371
CNN	0.8462	0.8248	0.8387	0.8366	0.0214
Bi-LSTM	0.8343	0.8271	0.8483	0.8365	0.0212
Transformer	0.8019	0.7370	0.7765	0.7718	0.0649

We chose ATIS because it is a similar task with far stronger class imbalance. The ranking changes. The CNN and Bi-LSTM tie, with all their seeds above every baseline and Transformer seed. That gap is the clearest result in this project: on harder, imbalanced data, sequence-aware models beat the bag of words, which they couldn't do on SNIPS. The Transformer does not share that advantage: it has the widest spread and overlaps the baseline. ATIS has far less training data than SNIPS, which may be part of the reason, but we did not test that. A caution on macro F1: rare classes with a handful of test rows carry the same weight as "flight", and at seed 42 all four models scored zero F1 on "meal", which has

one test row.

7 Inference cost

Model	Parameters	ms per sample	Peak process memory (MB)	Increase during inference (MB)
Baseline	25,704	0.0035	333.9	0.4
CNN	588,095	0.0566	342.6	7.6
Bi-LSTM	735,879	0.1118	351.5	15.4
Transformer	735,751	0.1272	355.0	17.3

Measured on Intel(R) Core(TM) i9-9980HK CPU @ 2.40GHz, 6 threads, PyTorch 2.2.2, batches of 64 at length 35, using the trained SNIPS checkpoints for all three seeds, with each measurement in a fresh process. Process memory includes the Python and PyTorch runtime, so the increase column compares the models better. The baseline is about 16 times faster than the CNN at a comparable SNIPS score, so on SNIPS the simple model is the sensible choice. The CNN costs about half as much time as the Bi-LSTM with the same ATIS score.

8 Limitations

(1) Three seeds on one split can’t support significance claims. (2) Tuning used one seed and the grid is coarse. (3) Test scores from our earlier runs were visible while we were revising the pipeline, including the baseline’s 0.18. Validation scores drove the final choice of settings and the validation underfit was visible on its own, but the final test set wasn’t untouched by us in the strictest sense. (4) Training ran on two devices: a Mac CPU and a Colab Tesla T4, recorded per run in results.json. All Transformer runs and the SNIPS baseline and standard CNN were on the CPU; the other final runs were on the T4. So some comparisons cross devices, which we would remove by rerunning everything on one. Cost was measured on one machine. (5) ATIS rare classes have very few test rows, so macro F1 there moves with a few predictions. (6) The explanations of why models differ are interpretations, not tested causes.

9 Contributions and references

Ranjay Salmon: project lead, baseline and shared evaluation. **Ankit Yadav:** convolutional model and BatchNorm ablation. **Ripan Deep Kour:** recurrent model and bidirectionality ablation. **Teja Ekkurthi:** attention model, positional encoding ablation and the ATIS check. The commit history records who committed each change.

AI assistance: Claude Code (Anthropic) helped review the code, implement the fixes, generate figures and draft this report. We reviewed the results. **References:** Kim (2014), Convolutional Neural Networks for Sentence Classification, EMNLP. Vaswani et al. (2017), Attention Is All You Need, NeurIPS. Coucke et al. (2018), Snips Voice Platform, arXiv:1805.10190. Hemphill et al. (1990), The ATIS Spoken Language Systems Pilot Corpus, DARPA Workshop. Hugging Face datasets benayas/snips and tuetschek/atis.